

Deploying a Containerized Application Using IaC

Project Implementation Report: build steps, issues resolved and final verification

Project	Calculator web app on AWS ECS Fargate, provisioned with Terraform, delivered by Jenkins CI/CD
Repository	github.com/amodarivindu/containerized-application-deploymenet-using-iac (branch: dev)
Prepared by	Amoda Rivindu
Date	30 September 2026
Status	Completed: pipeline green, application verified in the browser

Contents

1. Executive summary
2. Architecture
3. Technology stack and repository layout
4. Implementation steps followed
5. Design changes made during the project
6. Issues encountered and how they were fixed
7. CI/CD pipeline and final successful run
8. Scaling procedures
9. Operating procedures
10. Security, cost and recommendations

1. Executive summary

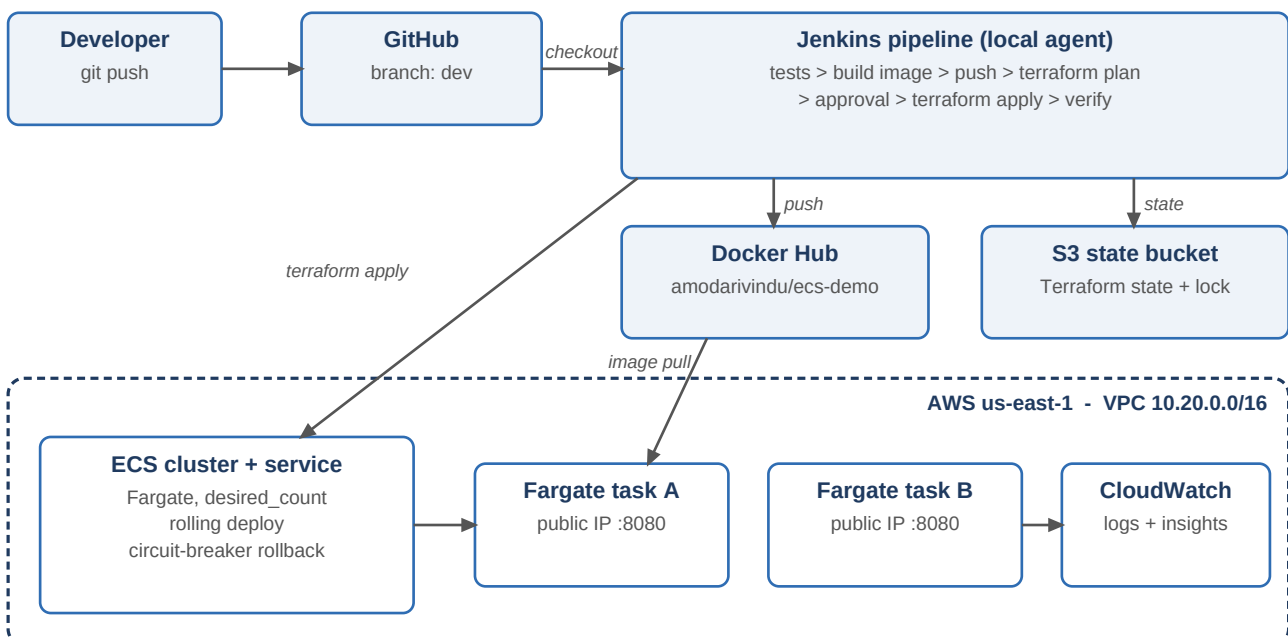
A Dockerized Python calculator web application was built and deployed to **AWS ECS Fargate**. All infrastructure is defined as code with **Terraform**, and every release is delivered by a **Jenkins** CI/CD pipeline that tests the code, builds the image, pushes it to **Docker Hub**, plans and applies the infrastructure change after a manual approval, and verifies the new version on every running task.

The pipeline completed successfully and the application was confirmed working in the browser. Along the way, ten problems were found and fixed. Most were environment issues: Jenkins running as a Windows service without a Unix shell, no administrator rights, S3 bucket naming and region conflicts, and a missing Jenkins plugin.

Delivered capabilities

- **Full CI/CD with Jenkins:** one pipeline with four actions: *deploy*, *plan-only*, *scale*, *destroy*.
- **Infrastructure as Code with Terraform:** VPC, subnets, security group, IAM roles, ECS cluster, task definition and service. 15 AWS resources in total, with remote state in S3 and state locking.
- **Vertical scaling:** task CPU and memory are pipeline parameters. A change creates a new task definition revision that ECS rolls out.
- **Horizontal scaling:** the service task count is a pipeline parameter, applied through an ECS service definition update.
- **Safety:** unit tests gate every build, a manual approval gates every apply, and the ECS deployment circuit breaker rolls back failed releases automatically.
- **Version-controlled codebase:** application, Dockerfile, Terraform, Jenkinsfile, helper scripts and setup documentation in one GitHub repository.

2. Architecture



Flow: a push to GitHub is picked up by Jenkins. The pipeline runs on a Jenkins agent under the developer's Windows account, so it can use Git's shell and Docker Desktop. The image is tagged `<git-sha>-<build>` and pushed to Docker Hub. Terraform, with its state in S3, updates the ECS task definition and service. ECS starts new Fargate tasks, which pull the image, pass their health check and replace the old tasks. Each task has a public IP and serves the app on port 8080. Container logs go to CloudWatch.

AWS resources

Terraform file	Resources	Purpose
network.tf	VPC, internet gateway, 2 public subnets, route table + 2 associations, security group	Network in 2 availability zones. Only port 8080 is open inbound.
iam.tf	Execution role (+ managed policy), task role	ECS pulls the image and writes logs. The app itself gets no AWS permissions (least privilege).
ecs.tf	Log group, ECS cluster, task definition, service	Runs the container. Health check on /health, rolling deploys with circuit-breaker rollback.
provider.tf	S3 backend, AWS provider, default tags	Shared remote state with lock file. Every resource tagged Project / Environment.
bootstrap/	S3 bucket (versioned)	One-time bucket that stores the Terraform state.

3. Technology stack and repository layout

Layer	Technology
Application	Python 3.12, Flask 3.1, Gunicorn 23; safe expression evaluator (AST whitelist, no eval)
Tests	pytest 8.4: 44 unit tests (calculator logic, security rules, HTTP API)
Container	Multi-stage Dockerfile: base > test (runs pytest) > runtime (non-root user, health check)
Registry	Docker Hub, public repository amodarivindu/ecs-demo
Infrastructure as Code	Terraform >= 1.10, AWS provider 6.x, S3 backend with native lock file
Compute	AWS ECS on Fargate (serverless containers), us-east-1
CI/CD	Jenkins (existing local installation) with a local inbound agent; declarative Jenkinsfile
Observability	CloudWatch Logs, ECS Container Insights

app/ terraform/ Jenkinsfile scripts/ docs/ jenkins-iam-policy.json	src/app.py, src/calculator.py, src/templates/index.html, tests/, Dockerfile provider.tf, variables.tf, network.tf, iam.tf, ecs.tf, outputs.tf, bootstrap/ deploy plan-only scale destroy get-app-urls.sh (task URLs), start-jenkins-agent.ps1 (agent as user) RUN-GUIDE.md, SETUP.md, APP-CODE-GUIDE.md, TERRAFORM-GUIDE.md,
---	--

4. Implementation steps followed

#	Step	What was done	Result
1	Application	Built the Flask calculator: UI, POST /api/calculate, /api/info (version + host), /health. Added a safe evaluator and unit tests.	44 tests pass
2	Containerize	Multi-stage Dockerfile; tests run inside the build; the runtime image runs as a non-root user. Built and ran it locally.	Image healthy locally
3	Registry	Created the Docker Hub repository and a Read & Write access token.	Push works
4	AWS access	Configured the AWS CLI; created an IAM user for Jenkins with a scoped policy (docs/jenkins-iam-policy.json).	Credentials ready
5	State bucket	Ran terraform/bootstrap to create the versioned S3 bucket ecs-demo-tfstate-400493233228.	Remote state ready
6	Terraform	Wrote the network, IAM and ECS resources; checked them with terraform fmt and terraform validate.	Configuration valid
7	Pipeline	Wrote the Jenkinsfile: tests, build, push, plan, approval, apply, verify. Parameters for action, CPU, memory, task count.	Pipeline defined
8	Jenkins	Created a local agent running as the Windows user, installed the AWS Credentials plugin, added the aws-deployer and dockerhub credentials, created the job.	Agent online

#	Step	What was done	Result
9	Deploy	Ran Build with Parameters with ACTION=deploy and approved the Terraform plan.	Pipeline green
10	Verify	The pipeline confirmed the new version on each task; the app was opened in the browser at http://<task-ip>:8080.	App working

5. Design changes made during the project

The first design was simplified step by step to match the project scope and the local environment:

Change	Reason
Amazon ECR replaced by Docker Hub	Project requirement. Public repository, no registry resources in AWS.
GitHub Actions removed; Jenkins only	The project uses Jenkins as the single CI/CD tool.
Sample page replaced by a calculator application	A meaningful app with real logic, an API and tests.
Terraform simplified	Removed advanced constructs (count/cidrsubnet, preconditions, optional private-registry logic). 645 to 321 lines, easier to read and explain.
Load balancer and autoscaling removed	Simpler and cheaper (about \$16/month less). Tasks are reached directly on their public IPs; scaling is controlled by pipeline parameters.
Bundled Jenkins container removed	An existing local Jenkins installation is used instead.
Container health check added to the task definition	Without a load balancer, ECS needs its own health check for the circuit breaker to detect bad releases.

6. Issues encountered and how they were fixed

Every issue below was reproduced, diagnosed from its error output, fixed, and confirmed by a follow-up run.

#	Symptom	Root cause	Fix applied	Status
1	docker run: <i>port 8080 already in use</i> ; with -p 8088:8088 the page did not load	Jenkins itself listens on localhost:8080. The app always listens on 8080 inside the container.	Map host 8088 to container 8080: -p 8088:8080	Fixed
2	terraform init: <i>Missing region value</i>	The S3 backend deliberately leaves out bucket and region.	Pass -backend-config for bucket and region (or use backend.hcl).	Fixed
3	terraform init: HTTP 301, bucket <i>iacproject</i> in eu-west-1	The bucket belongs to another AWS account in another region (not in aws s3 ls).	Do not use it; create our own state bucket.	Fixed
4	bootstrap apply: <i>BucketAlreadyExists</i> for ecs-demo-tfstate	S3 bucket names are global; the name was taken.	Unique name with the account ID: ecs-demo-tfstate-400493233228. Updated the Jenkinsfile and IAM policy.	Fixed
5	Jenkins: <i>Cannot run program "sh"</i>	Jenkins runs as a Windows service (LocalSystem) whose PATH has no Git shell.	Set the Shell executable. Adding Git to the system PATH needed admin rights (see #6).	Replaced by #6
6	No administrator rights to change the service or system PATH	Service settings and the machine PATH need admin rights.	Added a Jenkins agent that runs as the user (scripts/start-jenkins-agent.ps1) with Git's sh on its PATH; built-in node executors set to 0.	Fixed
7	Agent: <i>Handshake error</i> , no X-Remoting-Capability header	The agent secret was copied incompletely (52 of 64 characters).	Copied the full 64-character secret from the node page.	Fixed

#	Symptom	Root cause	Fix applied	Status
8	Pipeline: <i>No such DSL method 'aws'</i>	The AWS Credentials plugin was not installed.	Installed the plugin; recreated aws-deployer as an AWS Credentials type.	Fixed
9	terraform fmt -check would fail in Jenkins	Manual alignment edits did not match Terraform's formatter.	Ran terraform fmt -recursive; formatting is now checked before every commit.	Fixed
10	Jenkins log very long and hard to read	docker push progress lines and Terraform colour codes.	docker push --quiet; TF_CLI_ARGS_* = -no-color.	Fixed

Not a defect: a test request with the character × sent from Git Bash with curl arrived empty. The terminal was not sending UTF-8. The same request sent as a UTF-8 file returned the correct result (80), and the browser always sends UTF-8.

7. CI/CD pipeline and final successful run

Stage	What it does	Final run
Checkout	Clones the dev branch; image tag = <git-sha>-<build>	Ran on agent <i>local</i>
Unit Tests	docker build --target test runs pytest	44 passed
Terraform Init & Validate	S3 backend, fmt -check, validate	Backend configured, configuration valid
Build Image	Runtime image with APP_VERSION baked in	ecs-demo:d3cfe29-6
Push to Docker Hub	Login with token, push tag + latest, logout	amodarivindu/ecs-demo:d3cfe29-6
Terraform Plan	Plan saved and archived as tfplan.txt	15 resources to create
Approval	Manual gate before any change (always for destroy)	Approved
Terraform Apply	Applies exactly the approved plan	Infrastructure created
Verify Deployment	Waits for a stable service, checks /api/info on every task, prints URLs	New version live

Outcome: the pipeline finished successfully and the calculator was opened and used in the browser at <http://<task-public-ip>:8080>, with the deployed version shown in the page footer.

8. Scaling procedures

Type	How to trigger (Jenkins > Build with Parameters)	What happens
Vertical	ACTION=scale, TASK_CPU=512, TASK_MEMORY=1024	New task definition revision; new tasks start first, then the old ones stop.
Horizontal	ACTION=scale, DESIRED_COUNT=3	The ECS service desired count is updated; ECS starts or stops tasks to match.
New release	Commit and push, then ACTION=deploy	New image tag, rolling replacement, automatic rollback on failure.

Valid Fargate CPU/memory pairs: 256 with 512-2048 MiB; 512 with 1024-4096; 1024 with 2048-8192; 2048 with 4096-16384.

9. Operating procedures

Start the build agent (after each PC restart, with Docker Desktop running):

```
powershell -ExecutionPolicy Bypass -File .\scripts\start-jenkins-agent.ps1 -Secret <agent-secret>
```

Find the current application URLs. Task IPs change on every deploy or scale:

```
bash scripts/get-app-urls.sh ecs-demo-dev-cluster ecs-demo-dev-svc
# or: AWS Console > ECS > ecs-demo-dev-cluster > ecs-demo-dev-svc > Tasks > Public IP
```

Tear down everything to stop costs (about \$0.80/day while running):

```
Jenkins > Build with Parameters > ACTION=destroy > Approve
```

10. Security, cost and recommendations

Controls in place

- Secrets live only in the Jenkins credential store (aws-deployer, dockerhub) and are masked in logs; Docker Hub uses an access token and is logged out after each push.
- Least privilege: a scoped IAM policy for Jenkins; an empty task role for the application; the container runs as a non-root user.
- The calculator never uses eval(); input length, exponent and result size are capped.
- Terraform state is versioned in S3 and locked during runs; every change needs a reviewed plan and a manual approval.

Recommended next steps

- Add an Application Load Balancer when a stable URL is needed (task IPs change on each deploy).
- Move tasks to private subnets with HTTPS in front for production use.
- Rotate the Jenkins agent secret and the AWS access keys periodically; prefer short-lived credentials where possible.
- Commit and push the final Jenkinsfile log clean-ups to dev, merge dev into main, and run ACTION=destroy when the environment is not in use.